Raft Consensus Implementation

Advanced Database Management — Computer Science (A & B), 4th Semester

Project 02 Manual / *Languages: C or C++*

1 Problem Statement

You will implement the Raft consensus algorithm and use it to build a replicated key-value store that can tolerate the failure of a minority of its nodes. A Raft cluster is a group of three or five servers (this project uses five) that agree on a single, totally ordered sequence of commands. Clients send commands to any server; the cluster internally decides on an ordering that all servers eventually apply to their local state machine. As long as a majority of servers are alive and can communicate, the cluster continues to make progress. When servers crash, lose network contact, or restart, the remaining majority continues to accept commands and the crashed servers catch up on their own terms when they come back.

This is a harder project than Project 02 Option 1 (master-slave replication), and it is harder for a specific reason: Raft makes stronger guarantees. In Option 1, an acknowledged write can still be lost during a failover if the master crashed before replicating it, because the election rule is a heuristic ("whoever has the highest LSN"). Raft, by contrast, guarantees that every acknowledged command survives every possible sequence of failures and recoveries, as long as no more than a minority of the cluster is ever down simultaneously. Achieving this guarantee requires implementing several invariants very precisely, the log matching property, the election restriction, the commitment rule, and it is easy to write code that looks correct but subtly violates one of them. Debugging these bugs is unpleasant because they only manifest under specific, rare timings. Accept this up front and plan your time accordingly.

The Raft paper by Ongaro and Ousterhout (2014) is your primary reference. It is required reading before you begin Phase 1. This manual describes the parts of Raft you will implement and provides a structure for doing so, but you should treat the paper as authoritative whenever there is any question of interpretation. Figure 2 of the paper, a compact, one-page specification of the entire algorithm, is the single most important page you will read during this project. Print it and keep it next to your keyboard.

You are not implementing every feature of Raft. Cluster membership changes, log compaction by snapshotting, and the optimized probe-based log backtracking for fast catch-up are all out of scope (the first two are bonus). You are implementing leader election, log replication with the commitment rule, follower catch-up via the straightforward (slow) backtracking procedure, durable persistent state, and a key-value state machine on top. What remains is enough to run a working replicated database that tolerates up to two simultaneous failures in a five-node cluster.

2 What the Final Product Looks Like

When your project is complete, a user can start five server processes on five separate machines (or five containers, or five terminals — see Section 3 for deployment options), connect a client to any of them, and observe the cluster accepting commands, tolerating failures, and recovering correctly. A typical session looks like this.

```
# terminal 1 through 5: start all five nodes
$ ./raftkv --id 1 --port 6001 --raft-port 7001 \
           --peers 2@h2:7002,3@h3:7003,4@h4:7004,5@h5:7005 --data ./d1
[node 1] starting
[node 1] persistent state loaded: currentTerm=0 votedFor=none log=[]
[node 1] entering FOLLOWER state, term 0
[node 1] election timeout 347ms, waiting for leader
[node 1] election timeout expired, converting to CANDIDATE
[node 1] starting election for term 1
[node 1] sent RequestVote to all peers
[node 1] received vote from peer 3 (term 1)
[node 1] received vote from peer 4 (term 1)
[node 1] received vote from peer 2 (term 1)
[node 1] got 4 votes, becoming LEADER for term 1
[node 1] sending initial heartbeat
```

```
[node 1] ready for client commands on port 6001

# terminal 6: client
$ ./raftkv-cli h1 6001
connected to node 1 (LEADER, term=1)

> PUT x 1
OK (committed at index 1 in term 1)

> PUT y 2
OK (committed at index 2 in term 1)

> PUT z 3
OK (committed at index 3 in term 1)

> GET x
1

> \status
node_id:       1
state:         LEADER
term:          1
commitIndex:   3
lastApplied:   3
log length:    3
peers:
  node 2: matchIndex=3, nextIndex=4, up
  node 3: matchIndex=3, nextIndex=4, up
  node 4: matchIndex=3, nextIndex=4, up
  node 5: matchIndex=3, nextIndex=4, up

# now kill the leader and watch the cluster recover
# terminal 1:
^C

# terminal 2 (after a second or so):
[node 2] leader heartbeat missing 800ms
[node 2] election timeout expired, converting to CANDIDATE
[node 2] starting election for term 2
[node 2] granted vote from peer 3
[node 2] granted vote from peer 5
[node 2] got 3 votes, becoming LEADER for term 2
[node 2] sending initial heartbeat
[node 2] ready for client commands on port 6002

# client reconnects to new leader
> QUIT
$ ./raftkv-cli h2 6002
connected to node 2 (LEADER, term=2)
> GET x
1
> GET y
2
> PUT w 4
OK (committed at index 4 in term 2)
```

The important observation is the term number advancing from 1 to 2 across the failover, the preservation of all three committed values (`x`, `y`, `z`) despite the crash of the original leader, and the fact that after the crash only four nodes are running and the cluster still accepts writes (because four is a majority of five). If a second node were also killed — leaving only three out of five — the cluster would still accept writes. If a third were killed, leaving only two out of five, the cluster would correctly refuse writes until at least one node recovered, because no majority is possible. This last behavior is the one students most often get wrong by accident, so your chaos test must verify it.

3 Deployment Options

As with the replication project, the Raft server is agnostic about how it is deployed; only the peer list has to contain reachable addresses. Pick whichever of the following works for you.

3.1 Five Processes on One Machine

Open five terminals (or use `tmux` with five panes) and start five copies of the server on different ports, using `localhost` in the peer lists. This is the recommended development setup and what the grader will use by default.

3.2 Docker Compose

A `docker-compose.yml` with five services is a clean way to run the cluster on one machine with actual network isolation between nodes. `docker stop` / `docker start` give you clean kill-and-restart semantics for the chaos tests.

3.3 Five EC2 Instances

If you have AWS access and want to test on real networks, five `t3.micro` instances are enough. The security group must allow inbound TCP on both the client port and the Raft port between the instances. Use private IPs inside the VPC for the peer list.

3.4 Cluster Size Is Fixed at Five

No matter which deployment you pick, the cluster has exactly five nodes. Five tolerates two simultaneous failures (majority of three), which is the smallest cluster size at which the full set of interesting scenarios becomes testable (three nodes tolerate only one failure; anything larger is gratuitous). Do not support dynamic reconfiguration or clusters of a different size.

4 Raft in 60 Seconds

This section is a reminder of the algorithm's structure for students who have already read the paper. It is not a substitute for the paper and will not make sense if you have not read it. Reread this section after completing the paper.

A Raft cluster is a replicated state machine. Every server holds a log of commands, and every server applies its log in order to its local state machine. The crucial invariant that Raft maintains is: **if any two servers have applied the same log index, they have applied the same command.** In other words, the logs agree up to the committed portion.

At any moment, one server is the leader, the others are followers, and occasionally candidates exist briefly during elections. The leader serves all client commands. It appends each command to its own log and replicates the log entry to the followers. Once a majority of the cluster (including the leader) has persistently stored the entry, the leader marks it committed and applies it to the state machine, then responds to the client. Followers learn about new commits from subsequent heartbeats and apply entries to their own state machines.

Three safety properties keep this correct:

1. **Leader election safety.** At most one leader per term. Enforced by the rule that a candidate must receive votes from a majority, combined with the rule that each server votes for at most one candidate per term.
2. **Leader completeness.** A leader for term T contains every entry committed in any earlier term. Enforced by the election restriction: a candidate whose log is less up-to-date than the voter's log does not receive the vote.
3. **Log matching.** If two logs have an entry with the same index and term, then the logs are identical up to that entry. Enforced by the consistency check in `AppendEntries`: when a leader sends new entries, it also sends the index and term of the immediately preceding entry, and the follower refuses if it does not match.

If you internalize these three properties, the implementation becomes a straightforward translation of Figure 2 from the paper into code. If you try to implement the code without internalizing them, you will write something that looks correct on the happy path and fails in exactly the wrong way during recovery.

5 Persistent and Volatile State

Every server maintains state according to Figure 2 of the Raft paper. The state is divided into three parts:

5.1 Persistent State (written to disk before responding to RPCs)

- `currentTerm`: the latest term the server has seen (starts at 0 on first boot, increases monotonically).
- `votedFor`: the candidate id that received this server's vote in the current term, or `none` if not yet voted.
- `log[]`: the sequence of log entries. Entry indices start at 1. Entry 0 is a conceptual sentinel with term 0 that does not actually exist.

These three fields **must be written to disk and `fsync`'d before the server responds to any RPC that modified them**. If the server crashes and loses its notion of `currentTerm`, it can vote twice in the same term after restart — a direct violation of election safety. The durability of this state is non-negotiable.

5.2 Volatile State (lost on crash, rebuilt on restart)

- `commitIndex`: the highest log index known to be committed. Starts at 0.
- `lastApplied`: the highest log index applied to the state machine. Starts at 0.

These are volatile because they are recomputed safely during normal operation: a restarted follower learns the leader's `commitIndex` from the next `AppendEntries`, and it replays the portion of its log between `lastApplied` and `commitIndex` into its state machine during startup.

5.3 Volatile State on Leaders Only (rebuilt when a new leader is elected)

- `nextIndex[peer]`: for each follower, the index of the next log entry the leader intends to send. Initialized to `(leader's lastLogIndex) + 1` immediately after election.
- `matchIndex[peer]`: for each follower, the highest log index known to be replicated on that follower. Initialized to 0.

Leaders track these numbers per peer to manage log replication. `nextIndex` drives what gets sent in the next `AppendEntries`; `matchIndex` is the leader's certainty about what has been stored. They are rebuilt from scratch after every election because a new leader cannot trust the state of the previous one.

6 Log Entries and the State Machine

A log entry has exactly three fields:

- `term`: the term in which the leader that created this entry was leader.
- `index`: the position in the log (1-based).
- `command`: a byte string that is opaque to Raft and meaningful only to the state machine.

In this project, the command is a serialized key-value operation. Use the following binary format for commands (separate from the wire format for the log entry itself):

```
Byte 0:    op_type    1=PUT, 2=DELETE, 3=NOOP
Byte 1-2:  key_len    uint16 (0 for NOOP)
Byte 3-4:  value_len  uint16 (0 for DELETE and NOOP)
Bytes 5..: key bytes, then value bytes (neither null-terminated)
```

The `NOOP` command is a trick Raft leaders use immediately after election to commit an entry in their own term; see Section 7.4. It carries no data and does not modify the state machine.

The state machine is a hash map from string keys to string values. Applying a `PUT` inserts or overwrites; applying a `DELETE` removes the key; applying a `NOOP` does nothing. The state machine is completely deterministic: given the same sequence of committed entries, every server must arrive at the same key-value state. This determinism is the entire reason Raft's replicated log produces a replicated state machine.

7 The RPCs

Raft has two RPCs. That's all. Both are initiated by one server and responded to by another, and both are idempotent (sending the same request twice produces the same effect as sending it once, which is important because retries are inevitable on flaky networks).

7.1 RequestVote RPC

Sent by candidates to gather votes during elections.

```
RequestVote {
    term          uint64   candidate's term
    candidateId   uint64   candidate's id
    lastLogIndex  uint64   index of candidate's last log entry
    lastLogTerm   uint64   term of candidate's last log entry
}

RequestVoteResponse {
    term          uint64   current term for candidate to update itself
    voteGranted   bool     true means candidate received vote
}
```

Receiver implementation, per Figure 2:

1. Reply `false` if `term < currentTerm`.
2. If `term > currentTerm`, update `currentTerm`, persist it, convert to follower, and clear `votedFor`.
3. If `votedFor` is `none` or equals `candidateId`, **and** the candidate's log is at least as up-to-date as the receiver's log, grant the vote, persist `votedFor`, and reset the election timeout.
4. Otherwise reply `false`.

"At least as up-to-date" is defined by the election restriction: if the candidate's `lastLogTerm` is higher than the receiver's, the candidate is more up-to-date; if they are equal, the one with the higher `lastLogIndex` is more up-to-date; if both are equal, they are equally up-to-date.

7.2 AppendEntries RPC

Sent by the leader to replicate log entries and also as a heartbeat (with an empty entries array).

```
AppendEntries {
    term          uint64       leader's term
    leaderId      uint64       so follower can redirect clients
    prevLogIndex  uint64       index of log entry immediately preceding
                               the new entries
    prevLogTerm   uint64       term of that entry
    entries[]     LogEntry[]   new entries to store (empty for heartbeat)
    leaderCommit  uint64       leader's commitIndex
}

AppendEntriesResponse {
    term     uint64   current term, for leader to update itself
    success  bool     true if follower accepted the entries
}
```

Receiver implementation, per Figure 2:

1. Reply `false` if `term < currentTerm`.
2. If `term >= currentTerm`, accept the leader: update `currentTerm` if needed, persist, convert to follower, reset the election timeout.
3. Reply `false` if your log does not contain an entry at `prevLogIndex` whose term matches `prevLogTerm`. This is the log matching check.
4. If an existing entry in your log conflicts with a new one (same index but different term), delete that existing entry and all entries that follow it.
5. Append any new entries not already in the log.

6. If `leaderCommit > commitIndex`, set `commitIndex = min(leaderCommit, index of last new entry)`.
7. Reply `true`.

These steps translate to a dozen or so lines of straightforward code. The difficulty is not in any individual step; it is in making sure that *every* step is executed in *every* relevant case, without accidentally skipping the persistence write or the timeout reset in some branch.

7.3 Heartbeats Are AppendEntries

There is no separate heartbeat RPC. A heartbeat is simply an `AppendEntries` with an empty `entries` array. The leader sends these at a fixed interval (e.g., every 100 ms) to suppress elections. The receiving follower still performs the consistency check in step 3 — which means even a heartbeat can reveal log divergence and cause the follower to reply `false`, prompting the leader to back up its `nextIndex` for that follower and retry.

7.4 The No-Op Trick After Election

When a new leader is elected, it does not immediately know which entries from previous terms are committed. The rule for commitment (Section 9.3) only lets the leader commit entries from its *own* term. This means that if a previous leader crashed after replicating an entry to a majority but before committing it, the entry lives in the logs but is not yet "committed" from the new leader's perspective, and clients that were waiting on it will hang forever.

The fix is simple: immediately after winning the election, the leader appends a `NOOP` entry to its own log in the new term. Because this entry is in the leader's term, the commitment rule lets the leader commit it — and committing it, by the log matching property, transitively commits every earlier entry up to its index. Clients that were waiting on stale entries unblock.

Implement this. Failing to implement it does not break safety but does cause clients to hang after certain failure patterns, which is incorrect behavior.

8 Timing

Raft's safety does not depend on timing, but its *liveness* (the ability to elect a leader and make progress) does. Follow these guidelines:

- **Election timeout**: random, drawn from the range 300 to 500 milliseconds, resampled every time the timeout is reset. Randomization is essential: if all followers use the same timeout, they all become candidates simultaneously, split the vote, and time out again — a liveness failure. The random range ensures that one follower almost always times out first and wins before the others notice.
- **Heartbeat interval**: 100 milliseconds. Must be significantly less than the minimum election timeout so that a functioning leader never loses its leadership due to a spurious timeout.
- **Client request timeout**: if a client does not receive a response within 3 seconds, it should give up on the current server and try another. This timeout is in the client, not the server.

These numbers assume a local or LAN deployment. On a high-latency network (cross-region EC2, for example) they should be scaled up proportionally.

9 The Protocol, Step by Step

This section walks through the algorithm in the order you should implement it, integrating the RPCs and the state updates into a coherent picture.

9.1 Leader Election

A follower that has not received a valid `AppendEntries` within its election timeout converts to candidate. The candidate:

1. Increments `currentTerm` and persists it.
2. Votes for itself: sets `votedFor` to its own id and persists it.
3. Resets its election timeout (a fresh random value).
4. Sends `RequestVote` RPCs to all other servers in parallel.
5. Waits for responses. For each response, if the response's term is higher than the candidate's current term, the candidate steps down to follower. If `voteGranted` is true, the candidate counts it. If

the candidate has collected votes from a majority (three out of five, counting its own self-vote), it becomes leader.

If the election timeout expires while still a candidate (no majority yet), the candidate starts a new election in a yet higher term. This is what the random timeout prevents from looping forever.

When the candidate becomes leader, it immediately initializes `nextIndex` and `matchIndex` for every peer, appends a `NOOP` entry, and begins sending heartbeats to all followers. The heartbeats are `AppendEntries` RPCs; see the next subsection for how they eventually carry real entries.

9.2 Log Replication

When a client sends a command to the leader:

1. The leader appends the command to its own log as a new entry in the current term.
2. The leader persists its log (`fsync`).
3. The leader sends `AppendEntries` RPCs to all followers. For each follower, the RPC is constructed from the leader's `nextIndex[follower]`: it carries the entries starting from that index and a `prevLogIndex` / `prevLogTerm` pointing at the entry just before it.
4. For each follower that responds `success=true`, the leader updates `matchIndex[follower]` to the index of the last entry just sent, and `nextIndex[follower]` to `matchIndex + 1`.
5. For each follower that responds `success=false` (and whose response term is not higher — that would be a step-down), the leader decrements `nextIndex[follower]` and retries. This is the "backtracking" catch-up path: the leader keeps walking backward one entry at a time until it finds a common ancestor, then starts sending forward from there.
6. After updating `matchIndex`, the leader checks whether any log index is now replicated on a majority. If so, and if the log entry at that index is from the *current term*, the leader advances `commitIndex` to that index. The current-term restriction is what the `NOOP` trick exploits.
7. The leader applies committed entries to its state machine (from `lastApplied + 1` to `commitIndex`) and responds to the client for each applied command.

9.3 The Commitment Rule

A log entry is committed when all three of the following are true:

1. The entry exists in the leader's log.
2. The entry is stored on a majority of servers (`matchIndex >= entryIndex` on at least three nodes, counting the leader).
3. The entry's term is equal to `currentTerm` (the leader's current term).

The third condition is the subtlety that causes Section 7.4's `NOOP` trick to be necessary. Without it, a leader that saw a majority of followers containing an entry from a previous term might commit it — but the paper's Figure 8 describes a scenario in which this leads to a committed entry being overwritten, violating safety. Do not try to skip this rule. Implement it exactly.

9.4 Client Interaction

Clients do not need to know which server is the leader in advance. They can connect to any server and send a `PUT` or `GET`. If the receiving server is not the leader, it responds with an error containing the id of the current leader (which it learned from the last `AppendEntries` it received). The client then reconnects to that server. If the contacted server does not know who the leader is (for example, during an election), it responds with an error asking the client to retry shortly.

For `GET` operations, the safest behavior is to route them through the same log machinery as `PUT`: the leader appends a no-op read entry, waits for it to be committed, and only then returns the current value. This is slow but guarantees linearizable reads. A common optimization is to let the leader serve reads directly without going through the log, but this is only safe if the leader is certain it is still the leader (for example, by having just exchanged heartbeats with a majority). For this project, use the slow, log-based approach for reads; it is simpler to reason about and the performance difference does not matter at the benchmark scale.

10 Phase 1 — Single Node with Persistent Log and State Machine

The goal of Phase 1 is to build the substrate on which everything else will run. A single-process Raft node that maintains its persistent state on disk, accepts client commands, applies them to the key-value state machine, and survives a crash. No elections, no RPCs, no network. This phase is small and its purpose is to get the persistence, serialization, and state machine out of the way before the protocol logic lands on top.

Build these components:

- The persistent state file: a binary file containing `currentTerm`, `votedFor`, and the log. Define a format that supports appending to the log without rewriting the entire file (append-only log file, plus a small metadata file that holds the non-log fields). Use `fsync` after every update.
- The log entry serializer: convert a log entry struct to and from bytes. Include a CRC32 over each entry.
- The key-value state machine: a hash map with `apply(command)` and `get(key)` methods. The command serialization format is in Section 6.
- A TCP server that accepts client connections and handles `PUT`, `GET`, `DELETE`, `\status`, and `QUIT`. Since there is only one node and no Raft yet, the server simply appends commands to the log, applies them immediately to the state machine, and responds.
- Startup recovery: on launch, read the persistent state file, rebuild the log, replay committed entries into the state machine, and begin accepting clients.
- The client binary `raftkv-cli`.

At the end of Phase 1, the `\status` command should print the current term (always 0 in Phase 1), the log length, and `commitIndex`. Crashes and restarts should preserve all data.

11 Phase 2 — Elections and Heartbeats

The goal of Phase 2 is to bring up a five-node cluster that elects a leader and maintains leadership through heartbeats, but does not yet replicate client commands. A client that connects to a leader can issue `\status` and see the leader's role, but `PUT`s are rejected with an error saying log replication is not yet implemented.

Build these components:

- The RPC layer: a TCP connection between every pair of nodes, used for `RequestVote` and `AppendEntries`. Each RPC is a binary message with a header (RPC type, length) and a type-specific body. Use the message formats in Section 7.
- Background threads: one for each outgoing peer connection (sending RPCs) and one accepting incoming peer connections (receiving RPCs). Shared state is protected by a single mutex, held only briefly during each update.
- The election timer: a background timer that fires after a random interval in [300, 500] ms. Reset by any valid `AppendEntries` from the current leader or any granted `RequestVote`.
- `RequestVote` receiver logic, implemented per Section 7.1. Every transition of `currentTerm` or `votedFor` must be persisted (with `fsync`) before the reply is sent.
- The election procedure from Section 9.1: transition to candidate, increment term, vote for self, send `RequestVote` to peers, count responses.
- The leader's heartbeat loop: every 100 ms, send an empty `AppendEntries` to every peer. The heartbeat `AppendEntries` carries `prevLogIndex` and `prevLogTerm` matching the leader's current last log entry, so that followers can still detect divergence.
- `AppendEntries` receiver logic restricted to the heartbeat case (empty entries). Steps 1, 2, and 3 from Section 7.2 must be implemented and persisted; steps 4, 5, 6 are trivial when `entries` is empty.
- `\status` reporting the node's current role, term, and (if leader) the list of peers.

At the end of Phase 2, you should be able to start five nodes, watch them elect a leader within a few hundred milliseconds, kill the leader, watch a new leader be elected within another second or so, and kill nodes arbitrarily as long as a majority remains — a new leader will always emerge. Client commands are rejected but the protocol is visibly working.

11.1 Testing Phase 2 Thoroughly

Before moving on to Phase 3, spend real time testing Phase 2 under adversarial conditions. Run the cluster with 500+ election cycles by repeatedly killing the leader. Run it with artificial network delays (a few hundred milliseconds of sleep inside the RPC handler) to shake out timing-dependent bugs. Your election logic will have bugs; find them now, not after you have added log replication on top.

12 Phase 3 — Log Replication and Consistency

The goal of Phase 3 is full Raft: client commands are replicated via `AppendEntries`, committed via the majority rule, and applied to the state machine. Failover preserves all acknowledged commands. At the end of Phase 3 the full example session in Section 2 should work.

Build these components:

- The full `AppendEntries` receiver, implementing all six steps from Section 7.2.
- Per-peer `nextIndex` and `matchIndex`, initialized when a node becomes leader.
- The leader's send logic: for each peer, construct an `AppendEntries` with entries starting from `nextIndex[peer]`, send it, and handle the response. A `success=false` response with no higher term means decrement `nextIndex` and retry. A higher-term response means step down.
- The commitment rule from Section 9.3: after every `matchIndex` update, scan for the highest index replicated on a majority, and if that index's entry is in the current term, advance `commitIndex`.
- The apply loop: a background thread (or a synchronous call at the end of every `AppendEntries` receive and every commit advancement) that applies entries from `lastApplied + 1` to `commitIndex` into the state machine.
- The `NOOP` trick from Section 7.4: immediately on becoming leader, append a `NOOP` entry to the log.
- The leader-aware client redirect: a follower that receives a client `PUT`/`GET` replies with `NOT_LEADER leader=N` (or `UNKNOWN_LEADER` during an election), and the client reconnects.
- Log-based reads: a `GET` from a client is processed by the leader appending a read entry (or just a `NOOP`) to the log, waiting for it to commit, and then returning the value from the state machine at the time the entry is applied.

12.1 The Chaos Test

Phase 3 is evaluated primarily by a chaos test, which is the single most important deliverable. Write a test driver that runs the following scenarios in sequence, checking invariants after each step. Every scenario must pass.

1. Start all five nodes. Wait for a leader. Submit 1000 `PUT`s through the leader. Verify that all five nodes have exactly the same state machine content (connect to each and compare using `\status` or by reading every key).
2. Kill the leader. Wait for a new leader. Submit 1000 more `PUT`s. Restart the killed node. Verify that the restarted node catches up and ends with the same state as the others.
3. Kill two followers (keeping three alive, still a majority). Submit 500 `PUT`s. Restart the killed followers. Verify they catch up and all five nodes agree.
4. Kill three nodes at once (leaving only two alive). Attempt 100 `PUT`s; all should time out or fail, because no majority is possible. Restart one killed node, making three again; submit 100 `PUT`s; they should succeed.
5. Simulate a network partition by making two nodes refuse connections from the other three (and vice versa). The majority side should continue to accept writes. The minority side should have no leader and reject writes. Heal the partition: the minority side should re-elect (or re-join whoever is leader on the majority side) and catch up.
6. Repeat scenario 1 but kill the leader partway through the 1000 `PUT`s, then verify that the count of committed entries after recovery is within a small margin of 1000 and that there are no gaps.

The test driver must run as a shell script or separate binary (not Python). Log every action it takes and every response. Include the logs from your final successful run with your submission.

13 Deliverables and Submission

13.1 What to Submit

A single zip archive containing:

- The complete source tree of your implementation, including the server (`raftkv`), the client (`raftkv-cli`), and the chaos test driver.
- A `Makefile` (or `CMakeLists.txt`) that builds all binaries with a single command on a recent Ubuntu LTS.
- A `README.md` with build instructions, cluster startup instructions for each deployment option from Section 3, the list of supported client commands, and any known limitations.
- A `design.pdf` document of at most 15 pages describing: the server architecture, the RPC layer, the persistent state file format, the state machine implementation, the handling of each step of the two RPCs including which fields are persisted when, the commitment rule and why the current-term restriction is necessary, the `NOOP` trick, the catch-up backtracking logic, and a detailed walkthrough of each of the six chaos test scenarios including the observed behavior and any edge cases discovered during testing.
- A `chaos/` directory with the chaos test driver, the scripts that run it against the five-node cluster, and the logs from your final successful run.
- A `docker-compose.yml` or equivalent for whichever deployment option you tested with.
- A `tests/` directory with unit tests for: the log entry serializer and deserializer, the `RequestVote` receiver logic (feed in hand-crafted RPCs and verify responses), the `AppendEntries` receiver logic including the log matching check and the conflict-delete behavior, and the commitment rule.

13.2 Archive Naming

`Group[Number]_Project02_Raft.zip`, for example `Group17_Project02_Raft.zip`.

13.3 Demonstration

Each group will present a 30-minute live demonstration during the week following the submission deadline. The demonstration consists of three parts. First, a walkthrough of the architecture and the most subtle parts of the implementation (the commitment rule, the persistence ordering, the backtracking logic) using the design document. Second, a live startup of the five-node cluster followed by the full chaos test, with the instructor free to propose additional scenarios on the spot ("kill the leader, wait 200 ms, kill another node, what happens?"). Third, a short oral examination in which each group member will be asked to explain, without notes, a specific component of the implementation, with particular focus on the three safety properties (election safety, leader completeness, log matching) and how each is enforced by specific lines of code. All group members must be present. Missing the demonstration results in a zero for Phase 3.

14 Bonus Opportunities

Each of the following is independent. Bonus credit is awarded on top of the base grade up to a maximum of 20 percentage points. To claim bonus credit, your design document must include a dedicated section explaining which items you attempted and how you verified them.

14.1 Log Compaction by Snapshotting (up to 10%)

Implement the snapshot mechanism described in Section 7 of the Raft paper. When the log grows beyond a threshold, the state machine is serialized into a snapshot file, the log is truncated to discard entries covered by the snapshot, and a new `lastIncludedIndex` / `lastIncludedTerm` are recorded. The leader sends snapshots to followers that have fallen so far behind that the leader no longer has their required entries, via a new `InstallSnapshot` RPC. Your chaos test must include a scenario where a follower is killed, the remaining cluster submits enough entries to trigger log truncation, and the killed follower is restarted; the follower must receive a snapshot and catch up.

14.2 Pre-vote Protocol (up to 5%)

Implement the pre-vote optimization (Section 9.6 of the paper). Before a follower increments its term and becomes a candidate, it first sends a `PreVote` RPC to check whether it would win an election. This prevents a partitioned node from disrupting the cluster by repeatedly incrementing its term during

the partition. Demonstrate the improvement with a chaos test scenario that partitions a single node, waits several seconds, heals the partition, and shows that the isolated node's term has not advanced to artificially high values.

14.3 Fast Log Backtracking (up to 5%)

The base project uses the simple one-entry-at-a-time backtracking from Figure 2. Implement the optimized backtracking described at the end of Section 5.3 of the paper, where the follower's `AppendEntriesResponse` includes the term of the conflicting entry and the first index in its log for that term, allowing the leader to skip back by a whole term at a time. Demonstrate in your benchmark that catch-up of a very-behind follower is significantly faster with this optimization.

15 Constraints and Prohibitions

The Following Will Result in Loss of Credit

- Use of any existing Raft library or consensus framework. This includes but is not limited to `hashicorp/raft`, `canonical/raft`, `willemt/raft`, `etcd/raft`, `logcabin`, `braft`, `nuraft`, and their language bindings. The point of the project is to implement Raft yourself.
- Use of any existing RPC or serialization framework (gRPC, Thrift, Protobuf, Cap'n Proto, FlatBuffers). Raft RPCs must be implemented as direct reads and writes on plain TCP sockets using the binary formats you define.
- Skipping the persistence of `currentTerm`, `votedFor`, or new log entries before responding to an RPC. Every state change mandated by the algorithm to be "persisted before responding" must in fact be written to disk and `fsync`'d. Violating this can cause the cluster to elect two leaders for the same term after restarts, which is a textbook safety bug.
- Skipping the commitment rule's current-term restriction (Section 9.3). A leader that commits entries from previous terms based solely on majority replication is exhibiting the bug described in Figure 8 of the paper. A chaos test scenario will attempt to trigger this bug.
- Using a fixed, non-randomized election timeout. Liveness is at stake.
- Use of Python at any layer, including the chaos test driver. Tests and scripts must be written in C, C++, shell, or JavaScript.
- Submitting code whose commit history shows large, infrequent commits. Raft is notoriously difficult to implement in a single sitting; a git history showing incremental development is the expected shape of an honest attempt.

16 Required Reading

The first item is required reading before you begin Phase 1 and must be re-read before Phase 3. Everything else is reference material.

1. Ongaro, D. and Ousterhout, J. "In Search of an Understandable Consensus Algorithm." *Proceedings of USENIX Annual Technical Conference*, 2014. The Raft paper. Read Sections 1 through 5 carefully before Phase 1; Figure 2 is the specification you will implement. Reread Sections 5.4 (the safety argument) and 5.4.2 (the Figure 8 scenario) before Phase 3. If you read only one thing for this project, read this paper.
2. Ongaro, D. *Consensus: Bridging Theory and Practice*, PhD dissertation, Stanford University, 2014. The dissertation version of the Raft paper, with more detail on issues that the conference paper glosses over. Chapter 3 covers everything in the paper but at greater length; Chapter 4 covers cluster membership changes (out of scope here but useful background); Chapter 5 covers log compaction (relevant to the bonus).
3. "The Secret Lives of Data: Raft," interactive visualization at `http://thesecretlivesofdata.com/raft/`. Walk through it before starting. It takes about 15 minutes and gives an intuition for term numbers, election behavior, and log replication that the paper alone will not.
4. Howard, H. and Mortier, R. "Paxos vs Raft: Have We Reached Consensus on Distributed Consensus?" *Proceedings of the 7th Workshop on Principles and Practice of Consistency*, 2020. Optional. A short paper arguing that Raft and Paxos are more similar than the Raft paper suggests. Useful perspective after you have finished the implementation.
5. Kleppmann, M. *Designing Data-Intensive Applications*, Chapter 9 ("Consistency and Consensus").

Background reading on the broader context of consensus protocols. Useful as a big-picture refresher if you get lost in the details.

Penalty Structure

- Missing documentation: –10%
- Security vulnerabilities: –5% per issue
- Poor code quality: –5% to –15%
- Academic integrity violations: Zero grade

Contact & Support

Teaching Assistants:
Jalal Ahmed: `bscs23134@itu.edu.pk`
Khadijah Farooqi: `bscs23128@itu.edu.pk`

Office Hours: Monday & Thursday,
11:30 AM – 2:30 PM